

ARDUINO IDE - SEN0344 - V1.1

04/12/2025 - Lara Ribeiro

Descrição

Esta documentação acompanha o desenvolvimento do código para auxiliar no desenvolvimento do firmware para o ESP32-S3-DevkitC-1U e SEN0344 no Arduino IDE 2.3.6. A versão 1.n está destinada apenas à implementação do SEN0344, a versão 2.n vai ser usada para a integração com o modulo do sensor de temperatura e a versão final 3.n vai ser para implementar todos os outros modulos do colete.

Neste ficheiro são analisadas e explicadas as bibliotecas `DFRobot_RTU`, `Wire` e `DFRobot_BloodOxygen_S`.

"DFRobot_RTU.h"

Esta é a biblioteca criada pelo fornecedor do sensor (DFROBOT)

```
https://github.com/DFRobot/DFRobot_RTU/tree/master
```

Funcionalidades

Ela atua como uma camada de abstração (HAL - *Hardware Abstraction Layer*) para a implementação do protocolo Modbus RTU sobre interfaces série (UART). As suas principais responsabilidades no sistema são:

1. Encapsulamento e Formatação de Dados (Data Packing)

A biblioteca é responsável pela construção correta das tramas de comunicação (*frames*) exigidas pelo protocolo Modbus.

- **Descrição:** Converte os comandos de alto nível solicitados pelo utilizador (ex: ler temperatura) numa sequência de *bytes* hexadecimal estruturada.
- **Processo:** Organiza a mensagem seguindo a estrutura rígida do protocolo:

```
Endereço do Dispositivo (Slave ID) → Código de Função → Dados/Parâmetros → Checksum.
```

2. Verificação de Integridade via CRC (Cyclic Redundancy Check)

Implementa algoritmos de verificação de erros para garantir a fiabilidade da transmissão de dados em ambientes propensos a ruído eletromagnético.

- **Na Transmissão:** Calcula automaticamente o valor de CRC de 16-bits (baseado no polinómio padrão do Modbus) para cada pacote de dados gerado e anexa-o ao final da mensagem antes do envio.
- **Na Receção:** Ao receber uma resposta do sensor, a biblioteca recalcula o CRC dos dados recebidos e compara-o com o CRC enviado pelo sensor.
- **Resultado:** Se os valores não coincidirem, o pacote é considerado corrompido e descartado, prevenindo o processamento de leituras falsas ou dados errados.

3. Abstração de Registos e Endereçamento

Fornece uma interface simplificada para a manipulação direta do mapa de memória do dispositivo escravo.

- **Funcionalidade:** Disponibiliza métodos de alto nível, como `readHoldingRegister` e `writeHoldingRegister`, que abstraem a complexidade dos códigos de função do Modbus.
- **Operação:** O programador apenas necessita de indicar o endereço do registo alvo e o valor a ler ou escrever. A biblioteca gere internamente o acesso à memória, o tamanho dos dados (*word/byte*) e a interpretação da resposta do sensor.

Conclusão

Apesar de a biblioteca `DFRobot_RTU` ser a implementação do protocolo **Modbus RTU** e ser essencial para a comunicação via **UART/Serial** (pinos RX/TX), no âmbito deste projeto, onde a comunicação é realizada através do protocolo **I2C** (pinos SDA/SCL), esta biblioteca **não é ativamente utilizada** no *runtime*.

No entanto, mantemos a biblioteca instalada por uma questão de **dependência de compilação**. A biblioteca de alto nível do sensor (`DFRobot_BloodOxygen_S`) tem referências à `DFRobot_RTU` para o modo Serial e, para evitar erros de compilação, a `DFRobot_RTU` deve estar presente no ambiente de desenvolvimento, mesmo que o seu código seja ignorado na execução.

"DFRobot_BloodOxygen_S.h"

Esta é a biblioteca de controlo específica para o sensor SEN0344. É responsável pela gestão do ciclo de vida do sensor e pela aquisição de dados.

```
https://github.com/DFRobot/DFRobot_BloodOxygen_S
```

Funcionalidades:

1. **Comunicação:** Permite o controlo do sensor tanto via protocolo **I2C** como via **UART**, seleccionando automaticamente a camada de transporte adequada.
2. **Leitura de Dados Processados:** Recupera os valores finais de Frequência Cardíaca (BPM) e Oximetria (SPO2) já calculados pelo algoritmo interno do sensor.
3. **Gestão de Estados:** Controla o início e o fim da recolha de dados (`sensorStartCollect`, `sensorEndCollect`), permitindo poupança de energia quando o sensor não é necessário.

"Wire.h"

Esta biblioteca é a implementação para o protocolo de comunicação **I2C**, ou seja, esta biblioteca permite que o microcontrolador comunique com dispositivos periféricos através de apenas dois fios: **SDA** (Serial Data) e **SCL** (Serial Clock).

No contexto deste projeto, a biblioteca é utilizada para configurar o ESP32-S3 como **Dispositivo Master**, permitindo-lhe iniciar a comunicação e controlar o sensor (Slave).

Principais Funcionalidades Utilizadas

```
https://docs.arduino.cc/language-reference/en/functions/communication/wire/
```

A biblioteca gere a temporização dos bits, o endereçamento e os sinais de controlo (START/STOP) do protocolo. As funções essenciais para a operação do sistema são:

- **Wire.begin(SDA, SCL)**
 - **Função:** Inicializa o barramento I2C.
 - **No ESP32:** Ao contrário do Arduino padrão, no ESP32 esta função permite definir explicitamente quais os pinos GPIO a utilizar para SDA e SCL, oferecendo flexibilidade no *pinout*.
- **Wire.setClock(frequência)**
 - **Função:** Define a velocidade de transmissão do relógio I2C.
 - **Configuração:** Permite alternar entre o *Standard Mode* (100 kHz padrão e mais robusto) e o *Fast Mode* (400 kHz - mais rápido).
- **Wire.beginTransmission(endereço)**
 - **Função:** Inicia a preparação de um pacote de dados para enviar a um dispositivo escravo específico, identificado pelo seu endereço de 7 bits (ex: **0x57** para o sensor MAX30102).
- **Wire.write(dados)**
 - **Função:** Coloca bytes na fila de envio (buffer). Estes dados não são enviados imediatamente, mas sim armazenados temporariamente até o comando de fim de transmissão.
- **Wire.endTransmission()**
 - **Função:** Finaliza a transmissão. É neste momento que o microcontrolador envia efetivamente os dados armazenados no buffer e o sinal de STOP, libertando o barramento.
- **Wire.requestFrom(endereço, quantidade)**
 - **Função:** O Mestre solicita ao Escravo que envie um número específico de bytes. O Mestre gera o sinal de relógio para ler esses dados.
- **Wire.read()**
 - **Função:** Lê e retorna o próximo byte recebido do escravo que esteja disponível no buffer de receção.

Conclusão

Através da biblioteca **DFRobot_RTU** a comunicação é feita via UART, mas o sensor, mesmo que comunique através de I2C, precisa dela para funcionar; a biblioteca **Wire** permite que a comunicação seja feita através de I2C; e a biblioteca **DFRobot_BloodOxygen_S** permite aceder e controlar os dados enviados pelo sensor. No entanto, nenhuma destas bibliotecas permite aceder/controlar os valores definidos para: Samples per second, Pulse Width, Sample averaging ou Burst. Apenas é possível controlar o I2C clock. Para saber esses valores é

necessário enviar um email ao fabricante questionando-o, visto que os valores não estão mencionados no datasheet do sensor ou medir através de um osciloscópio.